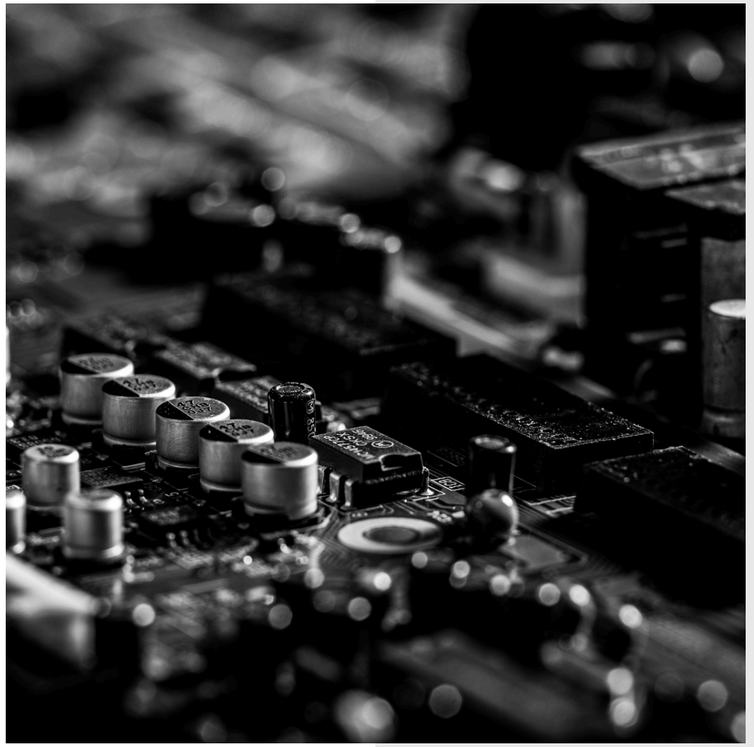
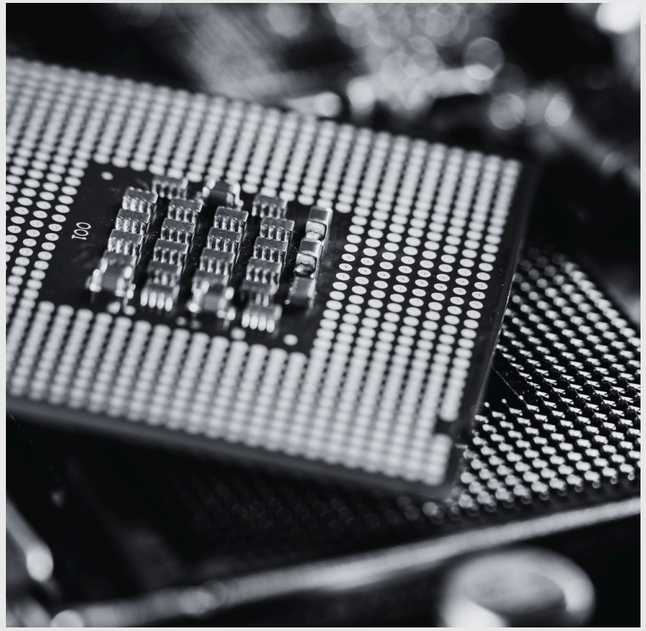
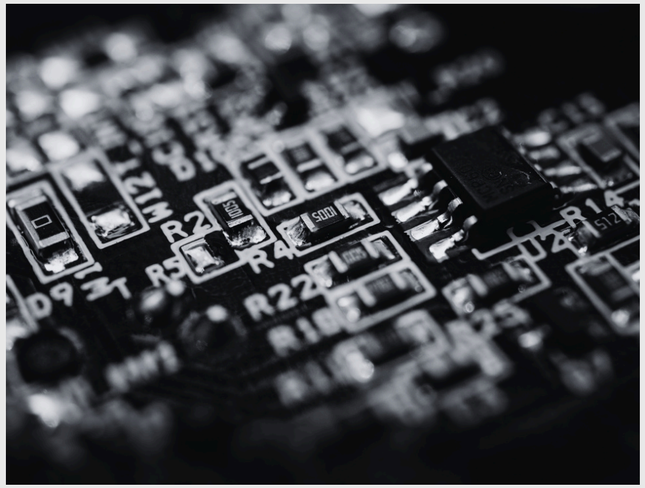
THE SMART ENVIROMENTAL LOGGER



Designer: Asma Abdi
Date: 10.2026







CONTENT

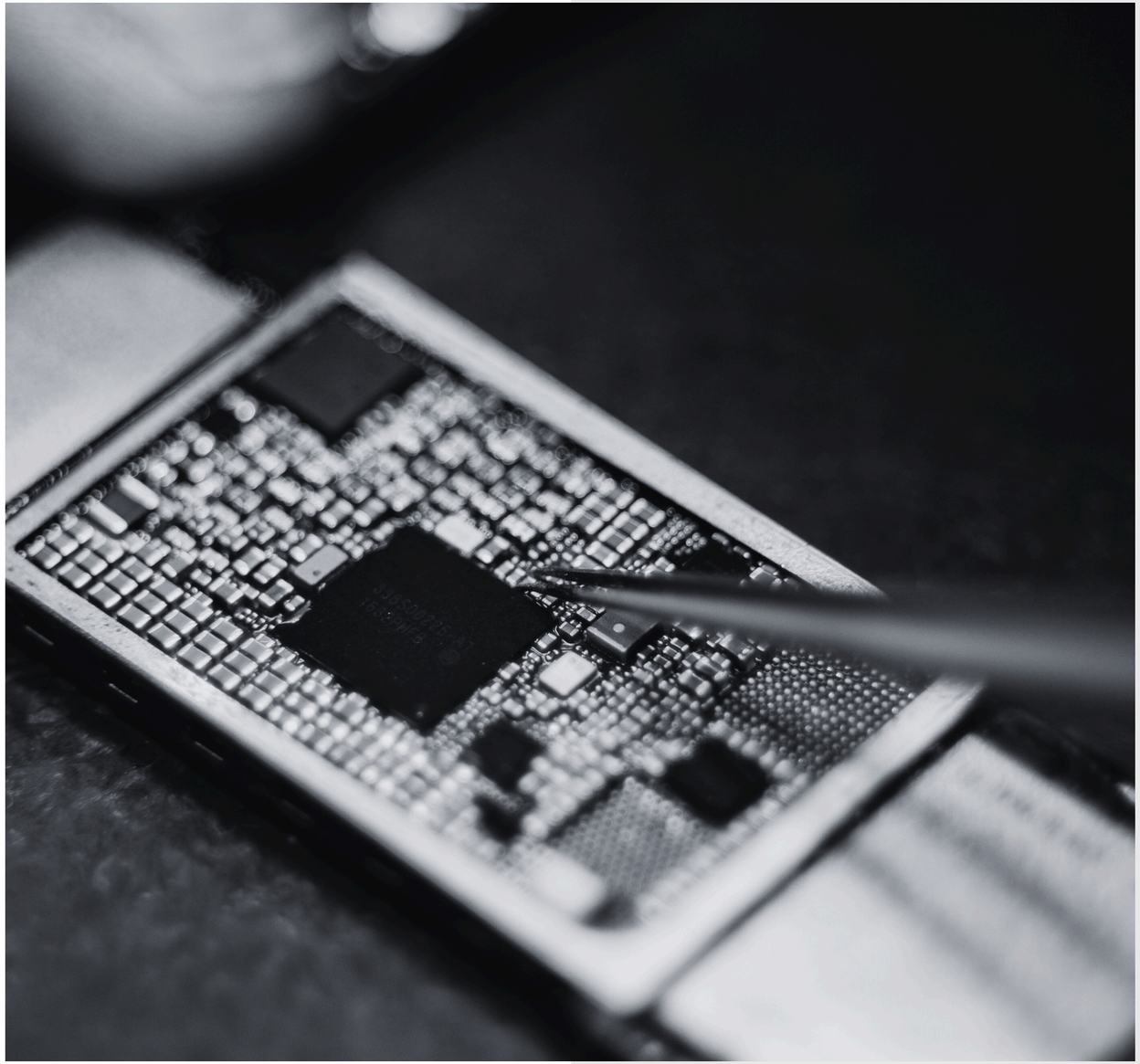
- Motivation & goal
- System overview
- System architecture (HW & SW)
- Functional Overview Abdul
- Software Modules developed
- Threads in the System Asma
- Thread communication & Data flow
- FSM purpose
- Conclusion & Lessons learned

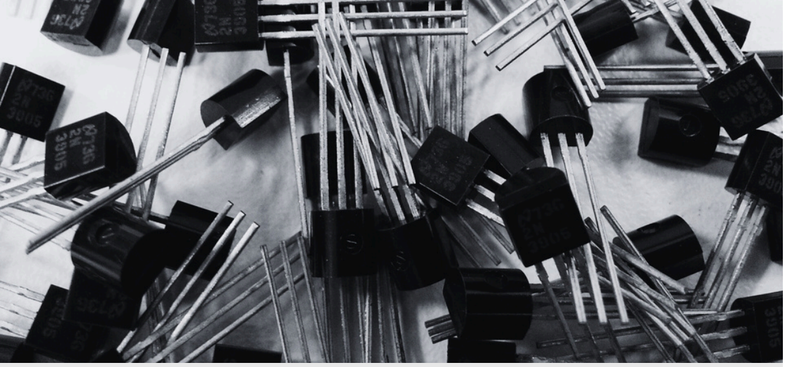
MOTIVATION & **GOAL**

- Temperature monitoring is critical in embedded systems
- Sensors must be read reliably and periodically
- Raw sensor data must be converted into meaningful system states

Goal:

- Measure temperature accurately
- Classify system behaviour
- Provide clear system feedback

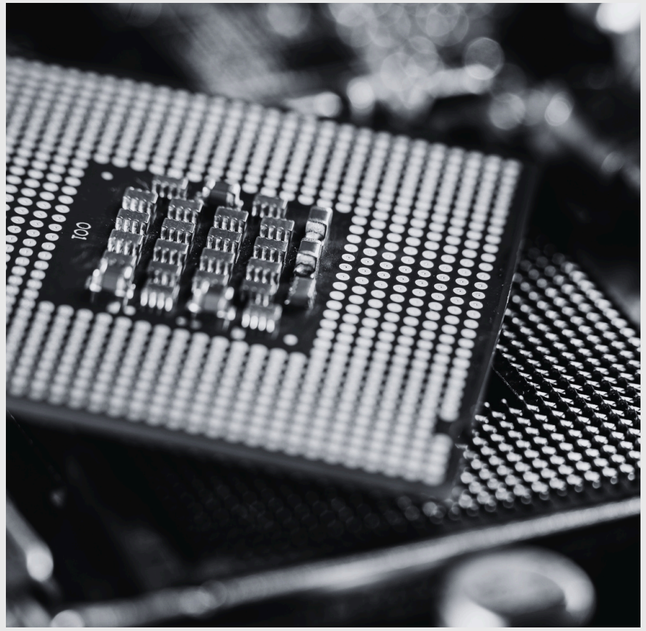


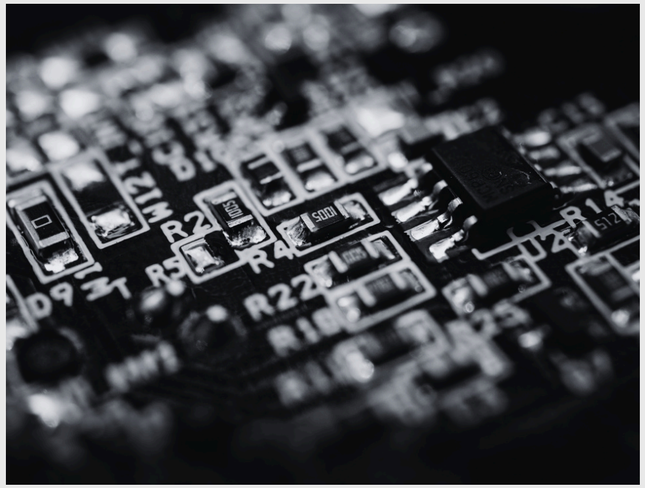




SYSTEM OVERVIEW

- Embedded system running on a microcontroller
- Uses an RTOS (OUTPOST)
- Reads temperature from a digital sensor
- Applies decision logic using an FSM
- Outputs results via development screen logging

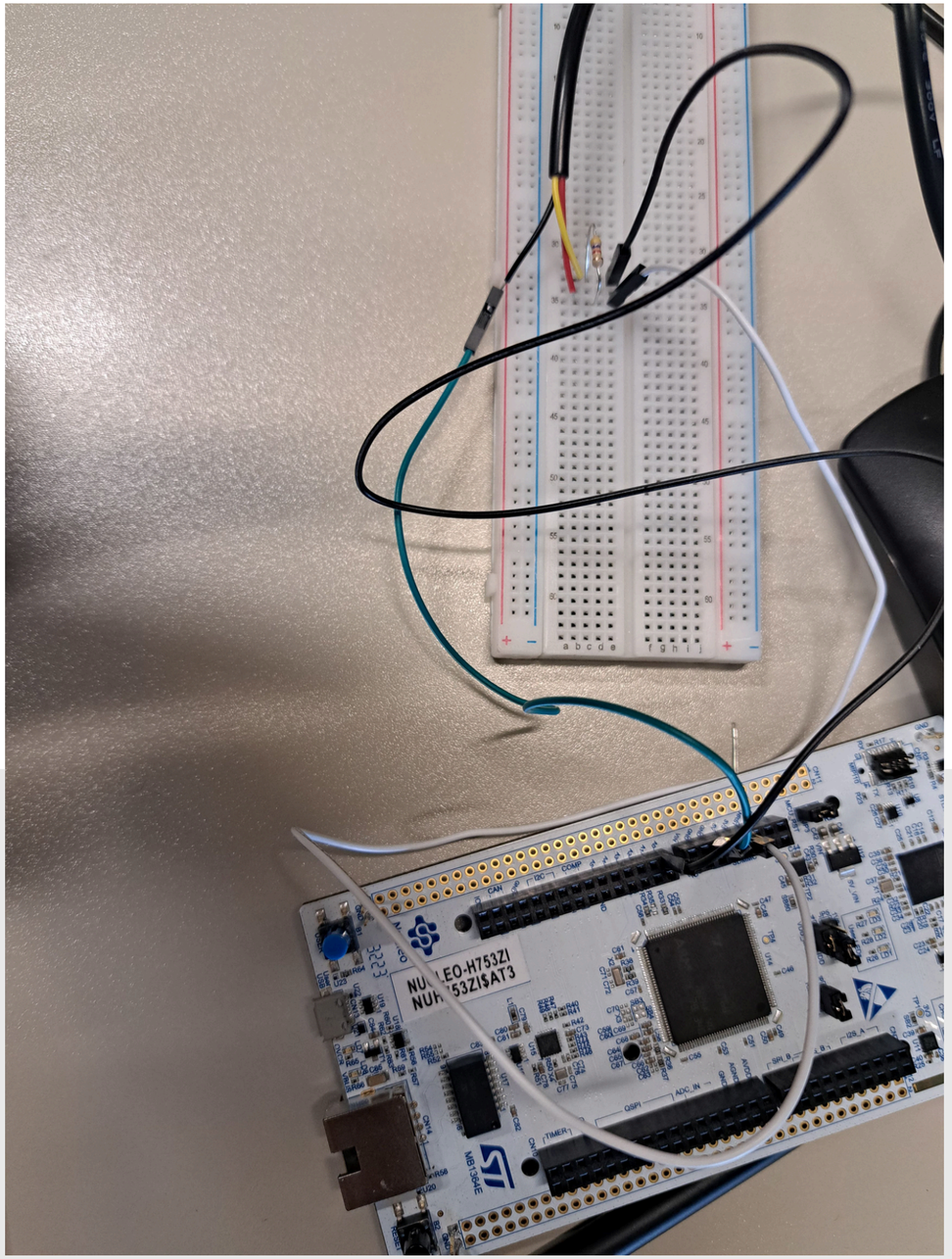


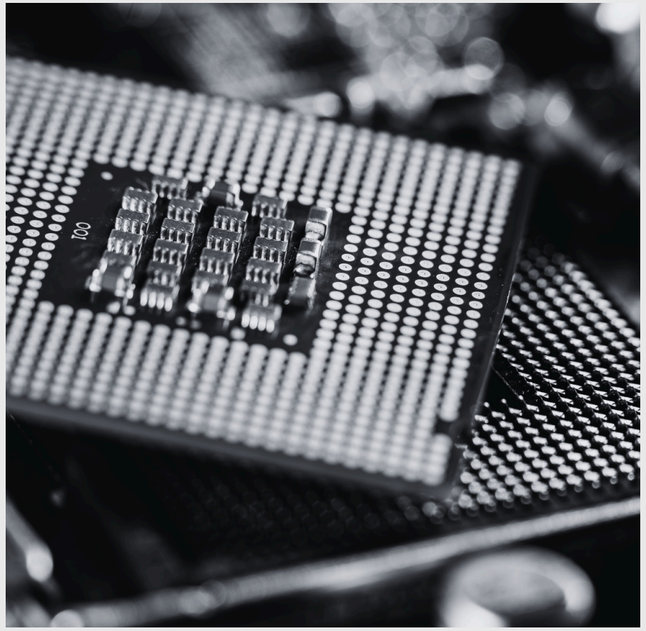


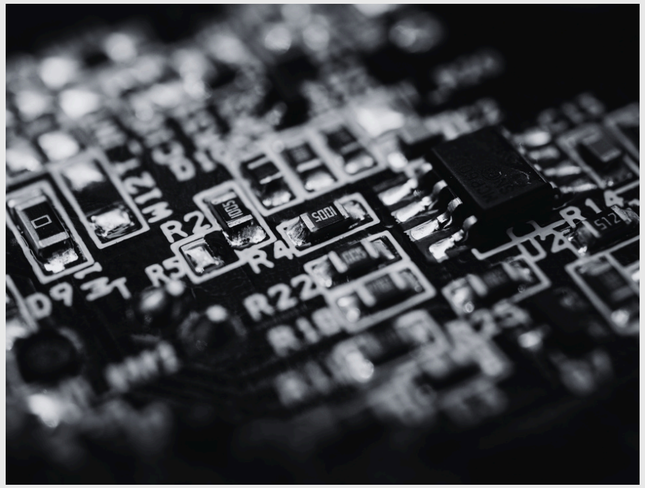
SYSTEM **ARCHITECTURE** ↗

HARDWARE	SOFTWARE
• One-Wire Temperature Sensor • STM32 Microcontroller • Pull-up Resistor	• HAL: Sensor communication and conversion • RTOS layer: Task scheduling and timing • Application layer: FSM logic

SET UP







SOFTWARE MODULES DEVELOPED

TEMPERATURE SENSOR

- Temperature conversion to °C

SENSOR TASK

- Periodic temperature sampling

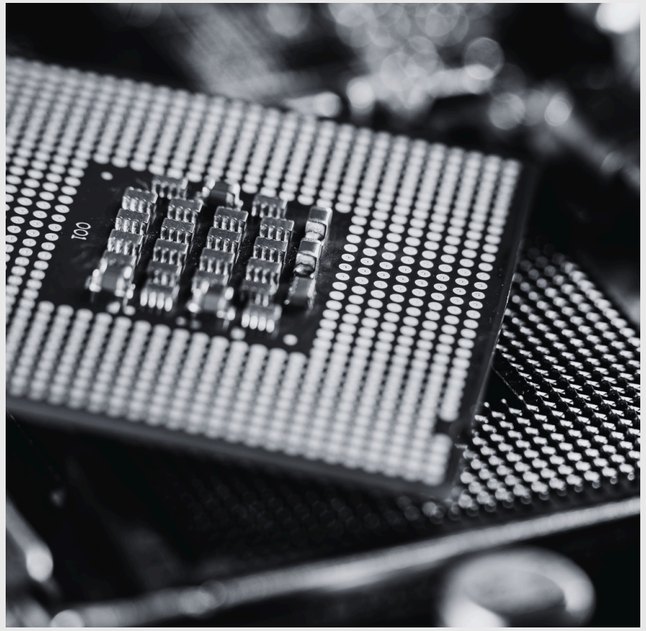
TEMPERATURE_MONITORING

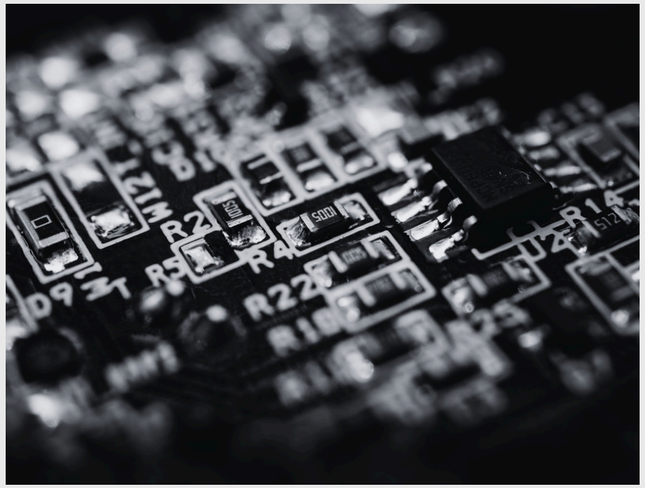
- FSM implementation

UNIT TEST

- Software testing

THREADS IN THE SYSTEM





IN SYSTEM

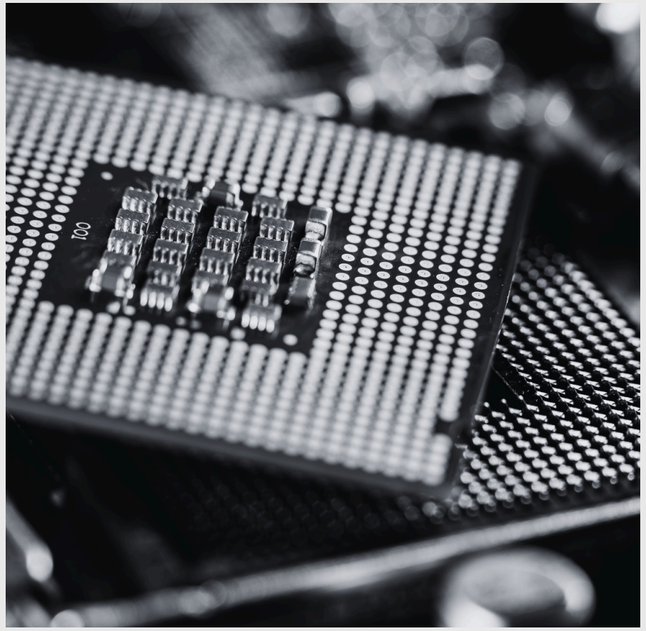
- The system uses OUTPOST RTOS threading
- Threads are created using outpost::rtos::Thread
- The RTOS scheduler controls execution

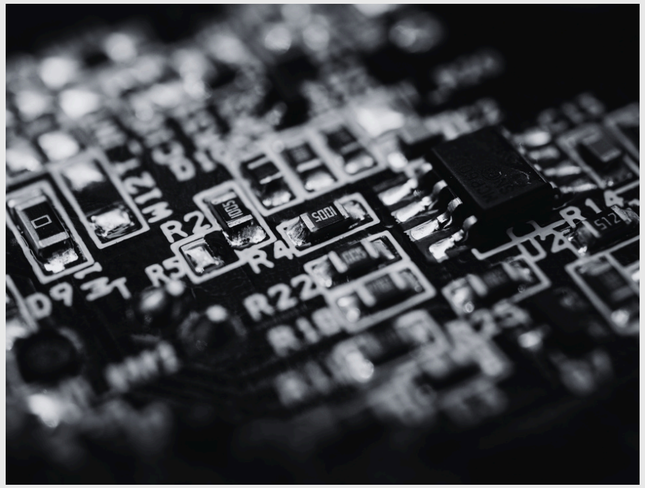
MAIN THREAD

- Acts as system entry point
- Initializes hardware drivers
- Initializes RTOS components
- Starts RTOS execution

MAIN THREAD

- Acts as system entry point
- Initializes hardware drivers
- Initializes RTOS components
- Starts RTOS execution





THREAD COMMUNICATION & DATA FLOW

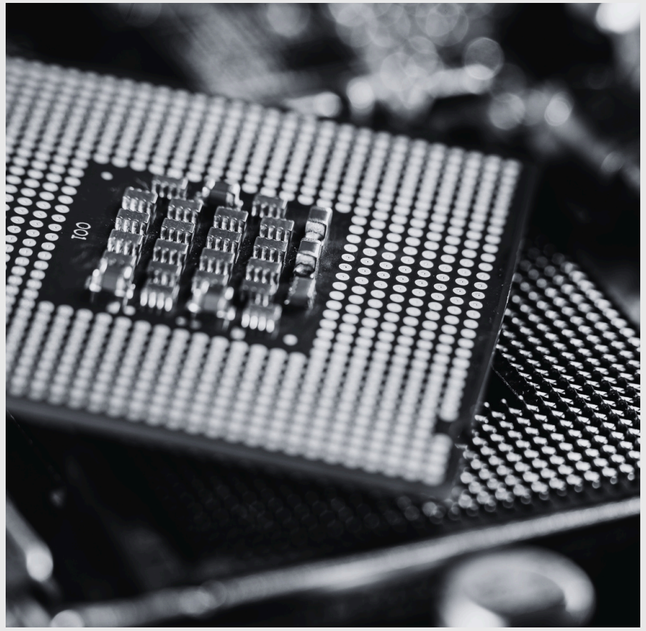
DATA FLOW

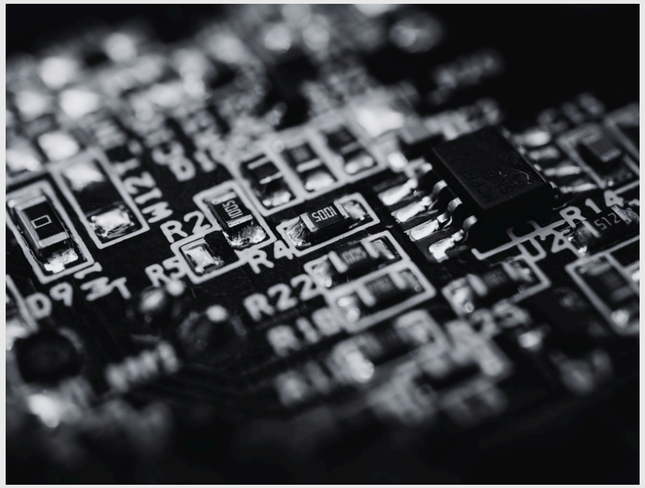
1. Temperature read via HAL / modm drivers
2. Sensor data processed in application code
3. State evaluation executed in same thread context
4. Output sent to logging interface

```
// HAL + modm DS18B20 driver
float DS18B20::getTemperature()
{
   startConversion();
   waitForConversion();
   return readRawValue() * 0.0625f; // Convert raw to °C
}
```

Sensor data is read via HAL/modm

1





THREAD COMMUNICATION & DATA FLOW

2. Sensor data processed in application code

```
float temperature = sensor.getTemperature();  // read from DS18B20
  fsm.updateState(temperature);               // process temperature
```

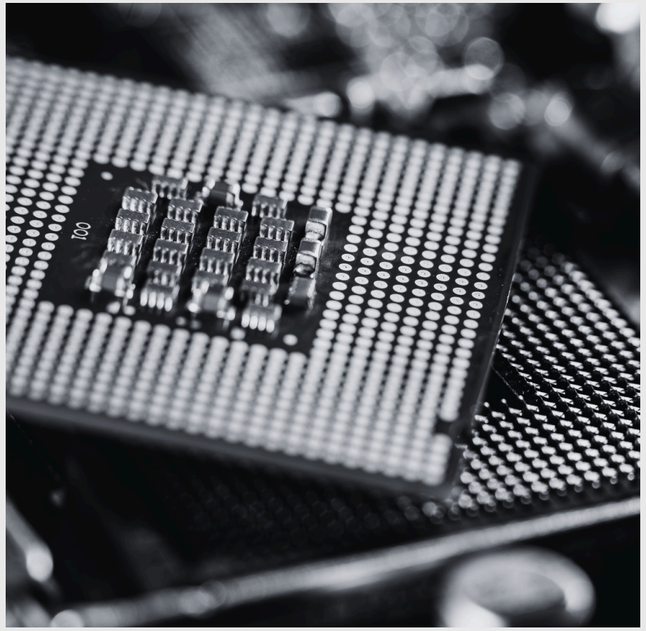
2

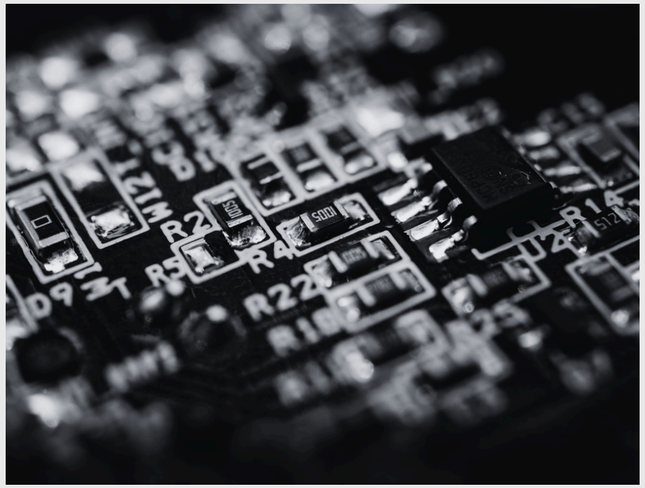
3. State evaluation executed in same thread context

```
void FSM::updateState(float temp)
{
    if (temp < WARNING_THRESHOLD)
        currentState = NORMAL;
    else if (temp < CRITICAL_THRESHOLD)
        currentState = WARNING;
    else
        currentState = CRITICAL;
}
```

3

FSM runs inside the same thread that reads the sensor; no inter-thread calls needed.





THREAD COMMUNICATION & DATA FLOW

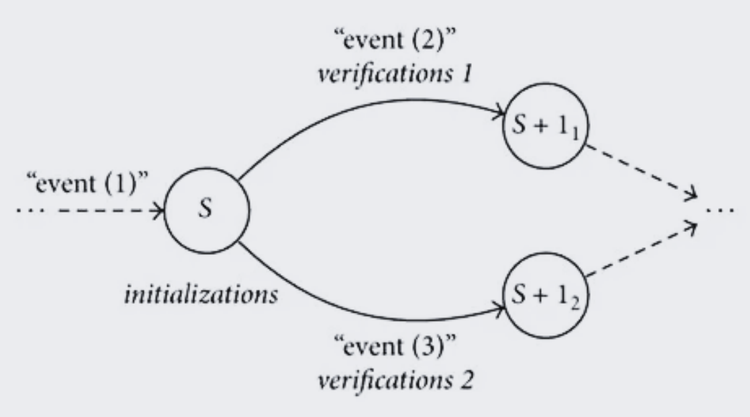
4

4. Output sent to logging interface

```
Logger::print("Temperature: %.2f °C, State: %s\n", temperature, fsm.getStateString());
```

- FSM state and temperature are logged; output handled in same thread

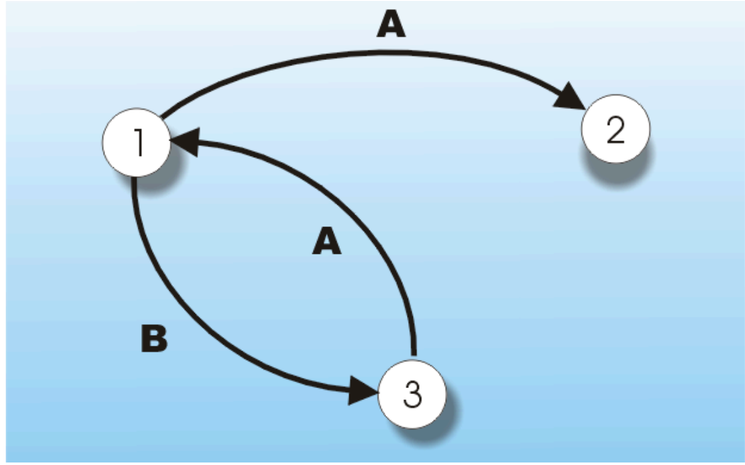
WHAT IS THE



PURPOSE OF **THE FSM**

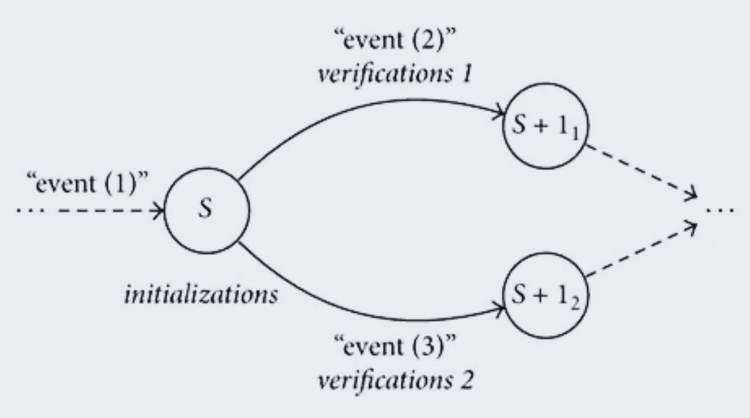
PURPOSE

- Convert raw temperature values into system states
- Central decision-making component
- Ensures deterministic system behavior



↗

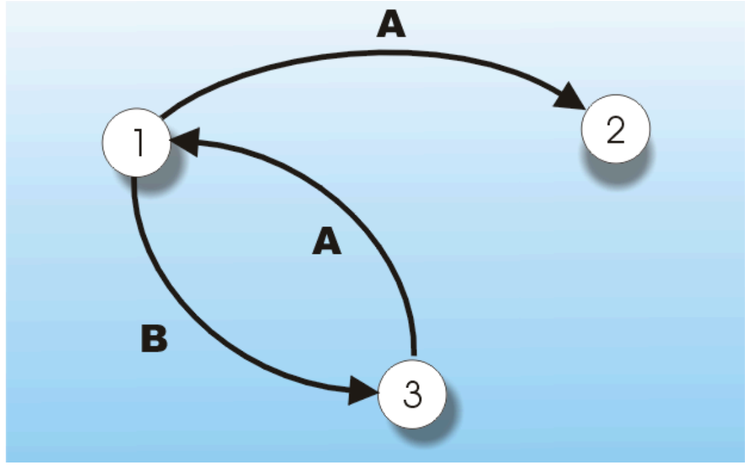
WHAT OUR



FSM
ACTUALLY DO?

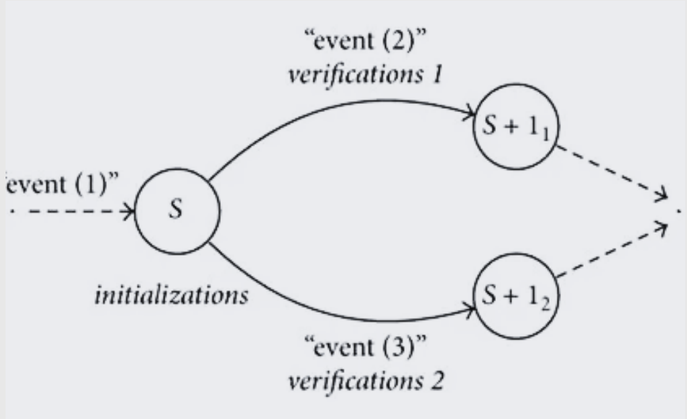
TASKS

- Here is a clear, slide-ready explanation of what your FSM does.
- Short bullets. No extra claims.
- What the FSM Does
- Receives temperature readings from the DS18B20 sensor
- Evaluates temperature against defined thresholds
- Determines the current system state
- Transitions between Normal, Warning, and Critical states
- Applies hysteresis to prevent state oscillation
- Outputs the current temperature and state to the development screen



↗

FSM



DESIGN & **STATES**

STATES

- NORMAL

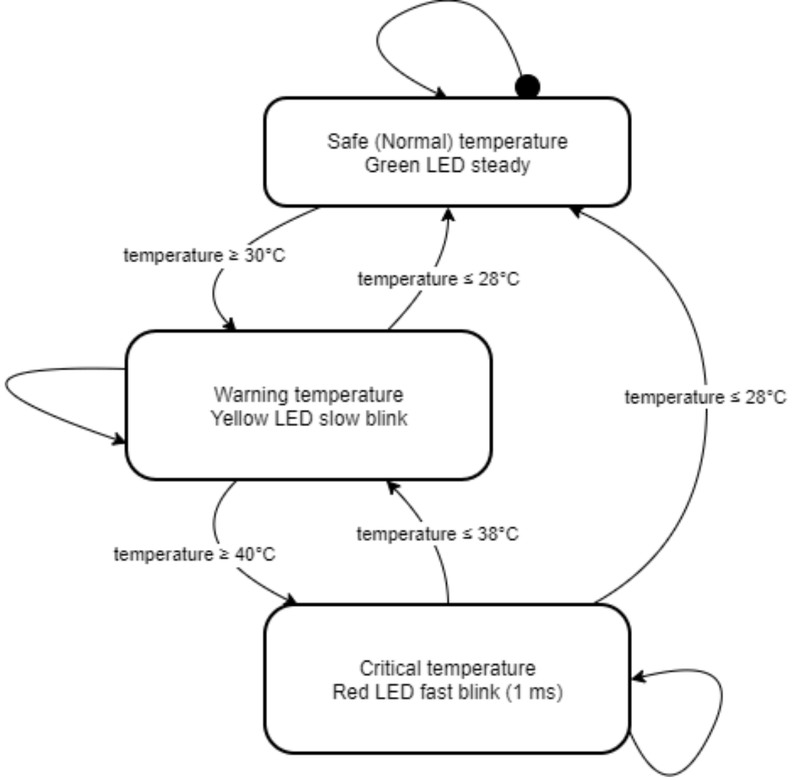
Temperature within safe range

- WARNING

Elevated temperature

- CRITICAL

Dangerous temperature level



Transitions & Hysteresis

- Transitions based on temperature thresholds
- Entry and exit thresholds are different
- Hysteresis prevents rapid state switching
- Results in stable and predictable behaviour

CONCLUSION

- Successfully implemented an RTOS-based monitoring system
- Clear separation of hardware, logic, and tasks
- FSM provides robust control behaviour
- System is easily extendable

CHALLENGES

STM31 CUBEMX

LESSONS LEARNED

- Correct interpretation of sensor data is critical
- FSM logic must not fix sensor errors
- Hysteresis is essential for real-world signals
- Logging is vital for debugging embedded systems
- Clean architecture simplifies debugging







THANK YOU

